



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ

Rai Technology University

(Estd. under Karnataka Act. No. 40 of 2013)

REPORT

Subject Title: DEEP LEARNING

Subject Code: 0CS351T

Submitted By:

Student Name: SATYAM KUMAR YADAV

University Enrolment Number: RTU24101CS013

Program Name: B.TECH CSE (AIML)

Semester: 5th sem

Session: 2026-2027

Submitted To:

Faculty Name: KUMAR SATYAM TANTI

Department of Computer Science & Engineering College Of

Engineering

Rai Technology University

AIM - To build and train a simple neural network using Tensorflow and Keras for classifying handwritten digits from 0 to 9 using the MNIST dataset, evaluate its performance, visualize training results, and compare the model performance after changing its architecture.

OBJECTIVES

- Load and explore the MNIST dataset
- Preprocess and normalize the images
- Build, train and evaluate a neural network
- Visualize accuracy and loss
- Predict five handwritten digits
- Compare the original model with a modified model using Dropout.

THEORY

MNIST is a dataset of 70000 handwritten digit images (60000 training and 10000 testing). Each image is a 28×28 grayscale image representing a digit from 0 to 9.

The images are normalized by dividing pixel values by 255.

$$X_{\text{new}} = \frac{X}{255}$$

A neural network is used to learn patterns from the images. The Flatten layer converts the 28×28

image into 784 values. Dense layers with ReLU learn features, while the final Softmax layer gives probabilities for the 10 digits.

The model is trained using the Adam optimizer and Sparse Categorical Cross-Entropy loss.

Accuracy is used to evaluate classification performance.

CODE

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
print("Image shape:", x_train[0].shape)
print("Pixel minimum:", x_train.min())
print("Pixel maximum:", x_train.max())
print("First 10 labels:")
print(y_train[:10])
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(x_train[i], cmap='gray')
```

```

plt.title(f"Label: {y_train[i]}")
plt.axis('off')
plt.tight_layout()
plt.show()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.summary()
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)

```

```
test_loss, test_accuracy = model.evaluate(  
    x_test, y_test, verbose=0)
```

```
print("Test Loss:", test_loss)
```

```
print("Test Accuracy:", test_accuracy)
```

```
print("Test Accuracy (%):", test_accuracy * 100)
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
```

```
plt.plot(history.history['val-accuracy'], label='Validation Accuracy')
```

```
plt.title('Training and Validation Accuracy')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(history.history['loss'], label='Training Loss')
```

```
plt.plot(history.history['val-loss'], label='Validation Loss')
```

```
plt.title('Training and Validation Loss')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
predictions = model.predict(x_test[:5], verbose=0)
```

```
predicted_labels = np.argmax(predictions, axis=1)
```

```
print("Actual Labels:", x_test[:5])
```

```
print("Predicted Labels:", predicted_labels)
```

```
plt.figure(figsize=(12, 3))
```

```
for i in range(5):
```

```
    plt.subplot(1, 5, i+1)
```

```
    plt.imshow(x_test[i], cmap='gray')
```

```
    actual = y_test[i]
```

```
    predicted = predicted_labels[i]
```

```
    plt.title(f"Actual : {actual} \n Predicted : {predicted}")
```

```
    plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
experiment_model = tf.keras.Sequential([
```

```
    tf.keras.layers.Input(shape=(28, 28)),
```

```
    tf.keras.layers.Flatten(),
```

```
    tf.keras.layers.Dense(128, activation='relu'),
```

```
    tf.keras.layers.Dropout(0.2),
```

```
    tf.keras.layers.Dense(64, activation='relu'),
```

```
    tf.keras.layers.Dropout(0.2),
```

```
    tf.keras.layers.Dense(10, activation='softmax')
```

```
])
```

```
experiment_model.compile (
```

```
    optimizer='adam',
```

```
    loss='sparse_categorical_crossentropy',
```

```
    metrics=['accuracy']
```

```
)
```

```
experiment_history = experiment_model.fit (
```

```
    x_train,
```

```
    y_train,
```

```
    epochs = 10,
```

```
    batch_size = 32,
```

```
    validation_split = 0.1
```

```
)
```

```
exp_loss, exp_accuracy = experiment_model.evaluate(
```

```
    x_test,
```

```
    y_test,
```

```
    verbose = 0
```

```
)
```

```
print("Experimental Test Loss:", exp_loss)
```

```
print("Experimental Test Accuracy:", exp_accuracy)
```

```
print("Experimental Accuracy (%):" exp_accuracy * 100)
```

```
print("Original Model Accuracy:", test_accuracy * 100, "%")
```

```
print("Dropout Model Accuracy:", exp_accuracy * 100, "%")
```

```
import pandas as pd
```

```
comparison = pd.DataFrame({
```

```
    "Model": ["Original Model", "Dropout Model"],
```

```
    "Test Accuracy": [
```

```
        test_accuracy * 100,
```

```
        exp_accuracy * 100
```

```
    ]
```

```
{})
```

```
comparison
```

```
plt.figure(figsize=(8,5))
plt.plot (
    history.history['val-accuracy'],
    label='Original Model'
)
plt.plot (
    experiment_history.history['val-accuracy'],
    label='Dropout Model'
)
plt.title ('Validation Accuracy Comparison')
plt.xlabel ('Epoch')
plt.ylabel ('Validation Accuracy')
plt.legend()
plt.grid()
plt.show()
```

OBSERVATION

In this experiment, Dropout layers with a rate of 0.2 were added after the hidden layers. Dropout randomly deactivates some neurons during training, which can reduce the model's dependence on particular neurons and help control overfitting. The original and modified models were trained using the same dataset, optimizer, batch size and number of epochs so that the effect of the architectural change could be compared more directly.

The original model achieved XX.XX% test accuracy, while the modified model achieved YY.YY% test accuracy. The training and validation curves were also compared to observe the effect of Dropout on generalization.

CONCLUSION

The MNIST handwritten digit classification problem was successfully implemented using Tensorflow/Keras. The dataset was loaded, explored, normalized, and visualized before training the neural network. A feed-forward neural network containing Flatten and Dense layers was designed and trained using the Adam optimizer and sparse categorical cross-entropy loss. The trained model achieved high classification accuracy on the unseen test dataset. Training and validation accuracy and loss were visualized to understand the learning behaviour of the model. The model was also tested on five handwritten digit images and the actual labels were compared with the predicted labels. Finally, an experiment was performed by adding Dropout layers, and its performance was compared with the original architecture.